

AI WEATHERWISE

Deliver Intelligent Weather Forecasts and AI-Powered Insights

NAAN MUDHALVAN PROJECT REPORT

Team Members	Rajalakshmi P
	Ramya M
	Arthi S
	Joshika M
	Madhusri R
Project Guide	Madhu Bursu
Department	B.Sc Computer Science with Artificial Intelligence
College Name	S.A. College of Arts & Science

Project: AI-powered weather information, summaries and personalized recommendations using Node.js, Express.js, MongoDB and Google Gemini.

1. Project Overview

AIWeatherWise is a RESTful backend application that provides current weather information and AI-generated weather insights. It retrieves weather data from an external weather service, stores user and favorite-location information in MongoDB, and uses Google Gemini to generate simple summaries and personalized recommendations.

1.1 Scenario-Based Case Study

Background

Alex is a travel enthusiast who frequently visits different cities and needs current weather information and useful suggestions about activities, clothing and outdoor plans.

Problem

- Difficulty tracking weather conditions for multiple cities.
- Raw weather metrics may be difficult to interpret.
- Need for personalized recommendations.
- Need to securely store favorite locations.
- Need for a convenient weather and AI API.

Solution

AI WeatherWise provides authentication, favorite-location management, weather retrieval and AI-powered recommendations. JWT protects user-specific operations, MongoDB stores application data, and Google Gemini converts weather information into natural-language insights.

Outcome

The system reduces the effort required to understand raw weather metrics by combining live weather data with AI-generated explanations and recommendations.

2. Software Requirements

Component	Requirement
Operating System	Windows 10/11, macOS or Linux
Runtime	Node.js v16 or above
Package Manager	npm v8 or above
Backend	Express.js
Database	MongoDB
ODM	Mongoose
Authentication	JWT
Password Security	bcryptjs
AI	Google Gemini / @google/genai
Weather Service	Weather API such as OpenWeatherMap
Testing	Postman or Thunder Client
Editor	Visual Studio Code

2.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i5 (8th Gen+) / AMD Ryzen 5 or equivalent

Component	Requirement
RAM	8 GB minimum; 16 GB recommended
Storage	At least 1 GB free project space
Internet	Required for external API requests

3. EPIC 1 – Project Architecture

AIWeatherWise follows a modular MVC-inspired architecture. Clients such as Postman or a frontend communicate with Express routes. Controllers coordinate business logic, services communicate with external weather and AI APIs, and Mongoose manages MongoDB persistence.

3.1 Components Definition

Component	Responsibility
Client / API Interface	Sends HTTP requests and receives JSON responses.
Express Server	Handles HTTP methods, routes, JSON parsing and CORS.
Authentication Middleware	Validates JWT tokens for protected endpoints.
Controllers	Process requests and prepare API responses.
Weather Service	Fetches current weather data.
AI Service	Uses Gemini to generate summaries and recommendations.
Database Interface	Mongoose communicates with MongoDB.
Error Handler	Provides consistent error responses.

3.2 ER Diagram Description

User Entity: _id, name, email, password (hashed), role.

Location Entity: _id, user, city, country, createdAt.

Relationship: One User can save multiple favorite Locations (one-to-many).

3.3 Key Features

- User registration and login.
- Secure password hashing.
- JWT authentication.
- Favorite-location CRUD.
- Current weather retrieval.
- AI weather summaries.
- Activity and clothing recommendations.
- Input validation and sanitization.
- Centralized error handling.
- Fallback behavior when optional external configuration is unavailable.

3.4 Roles and Responsibilities

Role	Responsibilities
Administrator	Monitor users, API health, logs and database operations.

Role	Responsibilities
Registered User	Manage favorite locations and request weather/AI insights.
Public User / API Consumer	Access permitted public weather information.

3.5 User Flow

1. User registers or logs in.
2. Server validates credentials and generates a JWT.
3. User saves a favorite city.
4. User requests current weather.
5. Backend retrieves weather data.
6. User requests an AI summary or recommendation.
7. Gemini generates the insight.
8. API returns the JSON response.

3.6 MVC Architecture Pattern

Layer	Description
Model	Mongoose schemas define User and Location data.
Controller	Handles requests, coordinates services and returns JSON.
View / API Interface	API routes act as the interface in this headless backend.
Service	Contains weather and Gemini integration logic.

4. EPIC 2 – Project Setup and Configuration

Create a folder named AI-WeatherWise, open it in Visual Studio Code and use the terminal.

Project Initialization

```
npm init -y
```

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

```
npm install --save-dev nodemon
```

Recommended Structure

```
AI-WeatherWise/
├── controllers/
├── models/
├── routes/
├── middleware/
├── services/
├── config/
├── utils/
├── index.js
├── .env
├── .gitignore
└── package.json
```

Environment Configuration

```
PORT=5000
```

```
MONGO_URI=your_mongodb_connection_string
```

JWT_SECRET=your_jwt_secret
GEMINI_API_KEY=your_gemini_api_key
WEATHER_API_KEY=your_weather_api_key

5. EPIC 3 – Backend Development

The backend uses Node.js, Express.js and MongoDB with separate routes, controllers, middleware, models and service modules.

5.1 Application Entry Point – index.js

index.js loads environment variables, initializes Express, configures middleware, connects to MongoDB, registers routes and starts the server.

5.2 Backend Modules

Module	Purpose
Authentication	Registration, login and JWT generation/validation.
User	Manage user information.
Location	CRUD operations for favorite locations.
Weather	Retrieve current weather.
AI Insight	Generate weather summaries and recommendations.
Middleware	Authentication, validation, sanitization and error handling.

5.3 API Endpoint Plan

Method	Endpoint	Purpose	Auth
POST	/api/auth/register	Register user	No
POST	/api/auth/login	Login and return JWT	No
GET	/api/weather/:city	Get current weather	As implemented
POST	/api/locations	Add favorite location	Yes
GET	/api/locations	Get saved locations	Yes
PUT	/api/locations/:id	Update location	Yes
DELETE	/api/locations/:id	Delete location	Yes
POST	/api/ai/insight	Generate AI insight	Yes

6. EPIC 4 – Database Configuration

MongoDB is used as the NoSQL database and Mongoose provides schemas, validation and database operations.

6.1 User Schema

_id: ObjectId
name: String
email: String
password: String (hashed)
role: String

6.2 Location Schema

_id: ObjectId
user: ObjectId → User
city: String
country: String
createdAt: Date

6.3 Database Connection

Mongoose uses MONGO_URI to establish the MongoDB connection. Successful connectivity should be confirmed before database operations.

6.4 Data Security

- Passwords are hashed before storage.
- JWT protects authenticated routes.
- Unique email validation prevents duplicates.
- Input validation reduces invalid data.
- API keys and database credentials are stored in environment variables.

7. EPIC 5 – Project Execution and API Testing

Before execution, verify that MongoDB is available and the .env file contains valid configuration.

7.1 Running the Backend

npm install

npm run dev

If configured in package.json, npm start can also be used.

7.2 Execution Flow

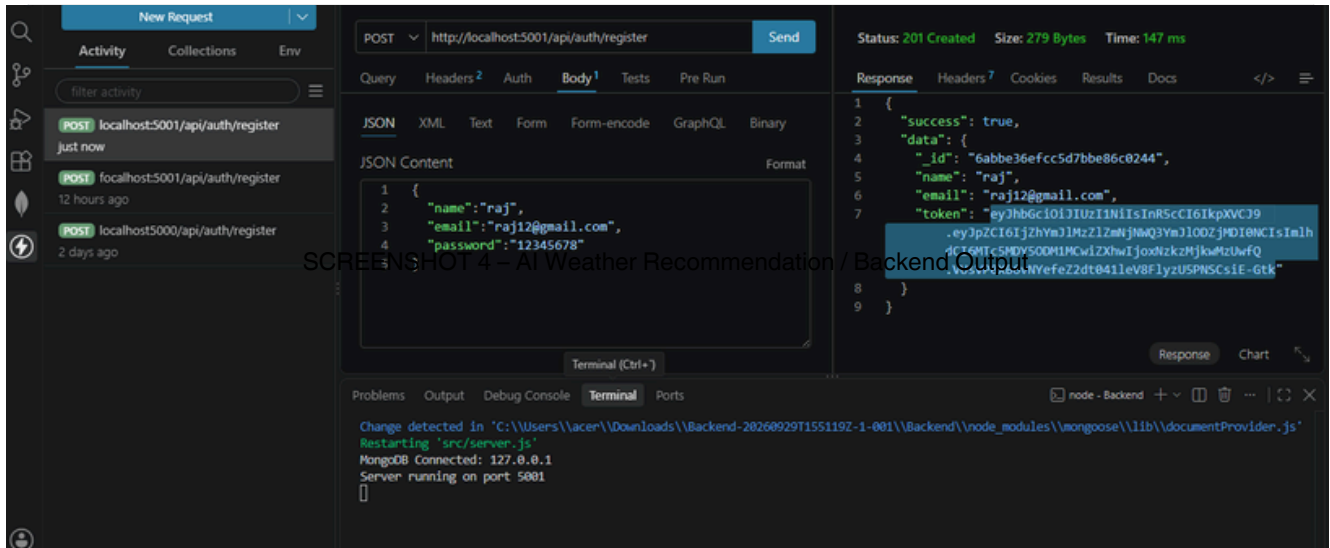
1. Load environment variables.
2. Initialize Express.
3. Register middleware.
4. Connect to MongoDB.
5. Register API routes.
6. Start the server.
7. Accept requests and return JSON responses.

7.3 API Testing

Test	Request	Expected Result
Register API	POST /api/auth/register	User created.
Login API	POST /api/auth/login	JWT returned.
Weather API	GET /api/weather/:city	Weather returned.
Favorite Location	POST /api/locations	Location saved.
Recommendation API	POST /api/ai/insight	AI recommendation returned.

8. Project Screenshots

The following four boxes are intentionally empty so the team can insert actual project screenshots.



9. Security and Reliability

- JWT authentication protects private routes.
- bcryptjs hashes passwords.
- API secrets are stored in environment variables.
- CORS is configured according to project requirements.
- Centralized error handling avoids unnecessary internal details.
- Validation and sanitization reduce malformed input.
- External API failures are handled gracefully.

10. AI Integration

Google Gemini converts relevant weather information into natural-language insights. Weather fields such as temperature, humidity, wind and conditions can be supplied to the Gemini service.

AI Processing Flow

1. Retrieve current weather.
2. Extract relevant metrics.
3. Build an AI prompt with weather context.
4. Send the prompt to Gemini.
5. Receive the generated response.
6. Return the summary or recommendation through the API.

Example AI output categories include weather summaries, suitable outdoor activities, clothing suggestions and basic travel precautions.

11. Project Outcome

AI WeatherWise demonstrates how REST APIs, database management, authentication and generative AI can be combined in a practical backend application. It helps users retrieve weather information, save preferences and understand conditions through AI-generated recommendations.

12. Future Enhancements

- Frontend dashboard.
- Interactive maps.

- Multi-day forecasts.
- Weather alerts and notifications.
- Personalized recommendation preferences.
- Weather history and analytics.
- Voice-based weather queries.
- Cloud deployment.
- Administrative dashboard.

13. Conclusion

AI WeatherWise is a practical AI-enabled weather backend combining Node.js, Express.js, MongoDB, secure authentication and Gemini AI. Its modular architecture supports maintainability and future expansion while demonstrating modern backend and generative AI techniques.

14. Project References / Resources

- Node.js documentation
- Express.js documentation
- MongoDB documentation
- Mongoose documentation
- Google Gemini API documentation
- Postman API testing documentation